TECNOLOGÍAS DE LA INFORMACIÓN

Desarrollo de Sistemas Informáticos

4 créditos

Profesor:

Ing. Roly Steeven Cedeño Menéndez, Mtr.

Titulaciones	Semestre
• *Tecnologías de la Información en Línea*	*Quinto*

NOMBRES Y APELLIDOS: WELLINGTON JOAO MUNOZ ALEJANDRO

PARALELO: "A"

SEMESTRE: Quinto

ACTIVIDAD 1: Estudio de caso - Sistema de Control de Rescate y Adopción Animal

Objetivo de la tarea:

PERIODO SEPTIEMBRE 2025 - ENERO 2026

Actividades a desarrollar en la Unidad _1_

Actividad # 1: Estudio de caso - Sistema de Control de Rescate y Adopción Animal

Objetivo de la tarea:

- Diseñar la arquitectura lógica de un sistema de gestión de rescate y adopción animal que permita aplicar correctamente patrones de diseño de software y flujos de trabajo profesionales (Gitflow), fortaleciendo la capacidad de análisis, abstracción y buenas prácticas en el desarrollo de soluciones escalables, reutilizables y mantenibles.
-

1. **Análisis del problema:** Un refugio animal necesita organizar la información de los rescates que recibe y coordinar el trabajo de sus responsables. Cuando los registros se encuentran dispersos, resulta difícil conocer el estado de cada animal, identificar quién lo atiende y dar seguimiento a su proceso.

   El sistema propuesto permitirá registrar casos de rescate, clasificar a los animales por especie, asignar voluntarios o veterinarios y actualizar el estado de cada caso. Además, notificará a los voluntarios cuando se registre un nuevo rescate.

Actores principales

Actor	Funciones dentro del sistema
Ciudadano o reportante	Reportar un animal que necesita rescate, indicando su ubicación y una descripción.
Voluntario o veterinario	Consultar los casos asignados y registrar avances del rescate o tratamiento.
Administrador del refugio	Revisar los casos, clasificar la especie, asignar responsables y supervisar los estados del proceso.

Requerimientos funcionales

- **RF01:** Registrar un caso de rescate con ubicación, descripción y datos del reportante.
- **RF02:** Clasificar al animal como perro, gato o fauna silvestre.
- **RF03:** Asignar un voluntario o veterinario responsable.
- **RF04:** Actualizar el estado del caso: Reportado, Rescatado, En Tratamiento y En Adopción. Para fauna silvestre, contemplar traslado o rehabilitación.
- **RF05:** Notificar a los voluntarios cuando ingrese un nuevo rescate.
- **RF06:** Consultar los casos y sus responsables.

Requerimientos no funcionales

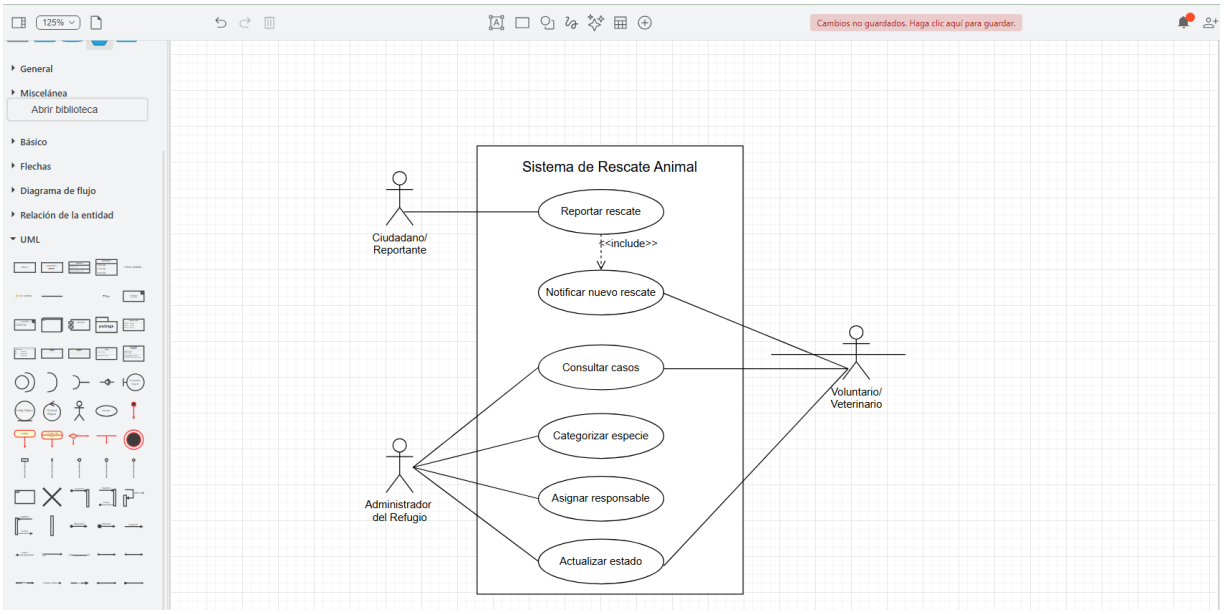
Requerimiento	Descripción
Escalabilidad	Permitir el crecimiento del número de casos y usuarios sin modificar toda la arquitectura.
Mantenibilidad	Separar las responsabilidades para facilitar correcciones y nuevas funciones.
Trazabilidad de versiones	Registrar los cambios mediante ramas y commits en GitHub.
Seguridad	Restringir las acciones según el rol de cada usuario.

2. Diseño del sistema

El sistema organizará el registro y seguimiento de los casos de rescate animal. Cada usuario tendrá funciones según su responsabilidad: el ciudadano reportará el rescate, el administrador coordinará su atención y el voluntario o veterinario registrará los avances.

2.1. Diagrama de casos de uso

El diagrama de casos de uso representa las acciones que cada actor puede realizar en la plataforma y permite comprender cómo participa en la atención de los animales.



El diagrama muestra las funciones de cada actor. El ciudadano reporta el rescate, el administrador coordina su atención y el voluntario o veterinario registra los avances. Al ingresar un caso, el sistema notifica a los voluntarios.

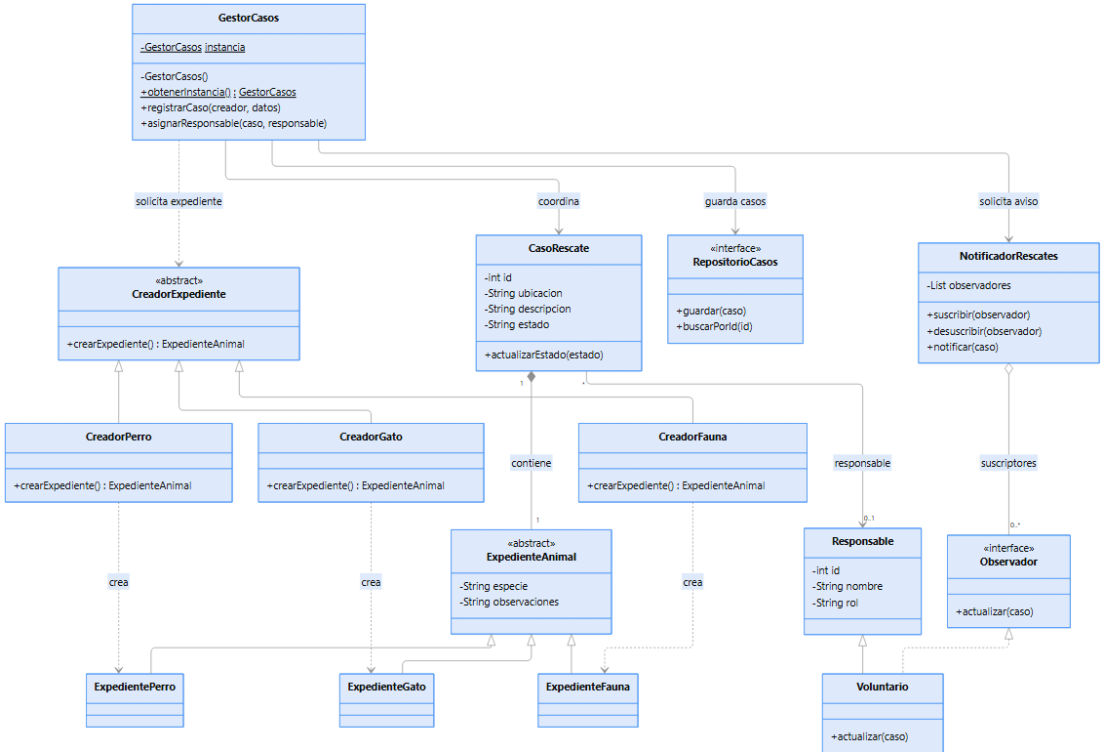
2.2. Patrones de diseño utilizados

Para organizar el sistema se utilizaron tres patrones de diseño: Singleton, Factory Method y Observer. Cada uno resuelve una necesidad específica del proceso de rescate.

Patrón	Aplicación en el sistema	Justificación
Singleton	La clase **GestorCasos** proporciona una única instancia para coordinar el registro de los casos dentro de la aplicación.	Permite centralizar esta coordinación y acceder al mismo gestor desde distintos componentes.
Factory Method	La clase abstracta **CreadorExpediente** define el método **crearExpediente().** Los creadores especializados lo implementan para generar expedientes de perro, gato o fauna silvestre.	Permite incorporar nuevos tipos de expedientes sin modificar los creadores existentes.
Observer	**NotificadorRescates** mantiene una lista de voluntarios suscritos y les avisa cuando se registra un nuevo caso.	Separa las notificaciones del registro del rescate y permite agregar o retirar suscriptores.

2.3. Diagrama de clases UML

El diagrama de clases representa los principales objetos del sistema y sus relaciones. El gestor coordina el registro, los creadores generan los expedientes según la especie y el notificador informa a los voluntarios suscritos.



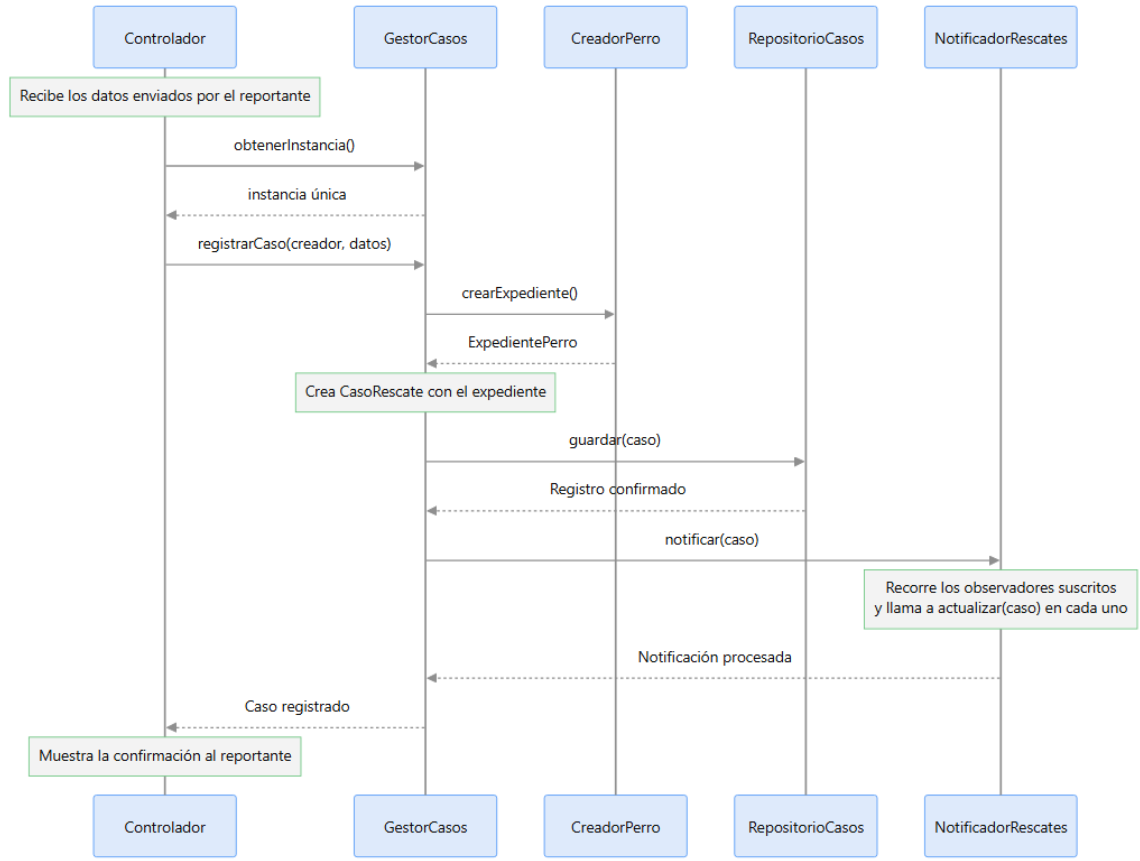
En Singleton, el constructor privado evita crear el gestor directamente desde otras clases. En Factory Method, cada creador concreto devuelve el expediente correspondiente a su especie. En Observer, los voluntarios implementan la operación **actualizar(caso)** para recibir avisos.

Aplicación de los principios SOLID

Principio	Aplicación
Responsabilidad única	La creación de expedientes, la gestión de casos, la persistencia y las notificaciones tienen responsabilidades separadas.
Abierto/cerrado	Una nueva especie puede incorporarse agregando un expediente y un creador especializados.
Sustitución de Liskov	Los creadores concretos deben respetar el contrato de **CreadorExpediente** y devolver un expediente válido.
Segregación de interfaces	La interfaz **Observador** contiene únicamente la operación necesaria para recibir notificaciones.
Inversión de dependencias	El gestor utiliza las abstracciones **CreadorExpediente** y **RepositorioCasos**; el notificador trabaja con **Observador**.

2.4. Diagrama de secuencia

El siguiente diagrama representa el registro de un rescate de perro. El sistema obtiene el gestor, crea el expediente correspondiente, guarda el caso y notifica a los voluntarios suscritos.

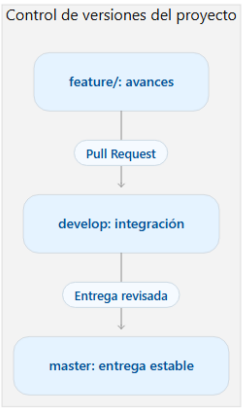


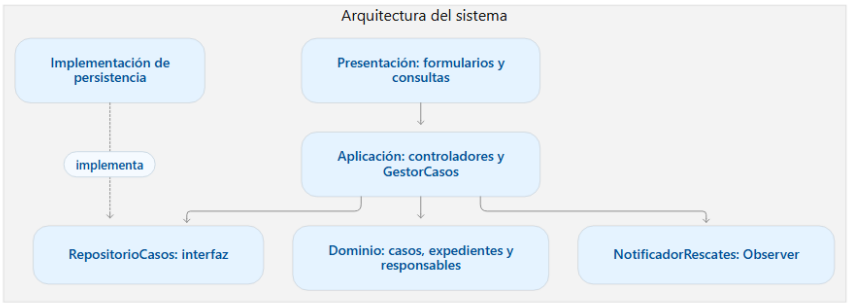
Singleton proporciona la instancia del gestor, Factory Method crea el expediente de perro y Observer comunica el nuevo caso a los suscriptores. La notificación se solicita después de confirmar que el caso fue guardado.

2.5. Arquitectura general del sistema

El sistema se organiza en cuatro partes: presentación, aplicación, dominio y persistencia. Esta separación permite modificar una parte con menor impacto sobre las demás.

Parte	Responsabilidad
Presentación	Mostrar formularios y pantallas para reportar y consultar rescates.
Aplicación	Recibir las solicitudes y coordinar las operaciones mediante controladores y el gestor.
Dominio	Representar los casos, expedientes, responsables y reglas del proceso.
Persistencia	Guardar y consultar los casos mediante una implementación de **RepositorioCasos**.





Las ramas de Gitflow permiten controlar los cambios en los documentos y componentes del proyecto. Este flujo forma parte del desarrollo y no de la ejecución del sistema.

3. Gitflow y control de versiones

Gitflow permite organizar los avances del proyecto y mantener un historial de los cambios. La rama **master** conserva la entrega estable, mientras que **develop** reúne los avances revisados. Las ramas **feature**/ se utilizan para desarrollar partes específicas antes de integrarlas.

Los commits describen los cambios realizados y los Pull Requests permiten revisar su integración. Esto facilita identificar cómo evolucionó el diseño y recuperar versiones anteriores cuando sea necesario.

Enlace al repositorio:

https://github.com/Joaomunoz6552/Actividad1-U1

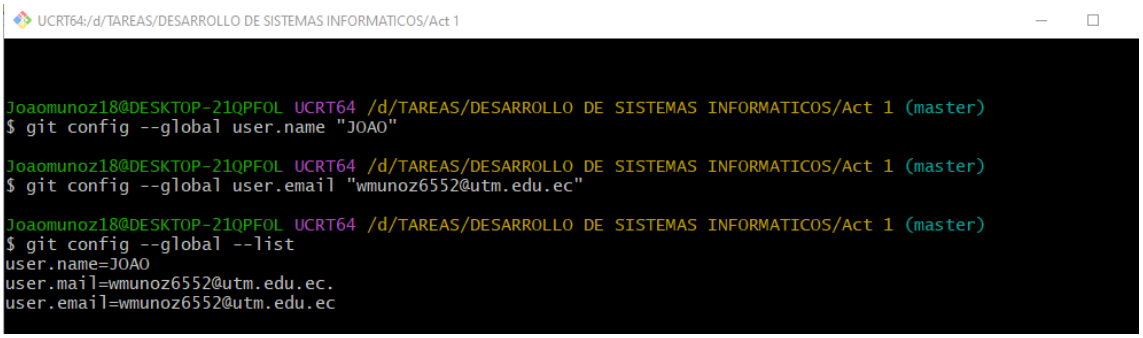
Evidencias

Enlace al repositorio:

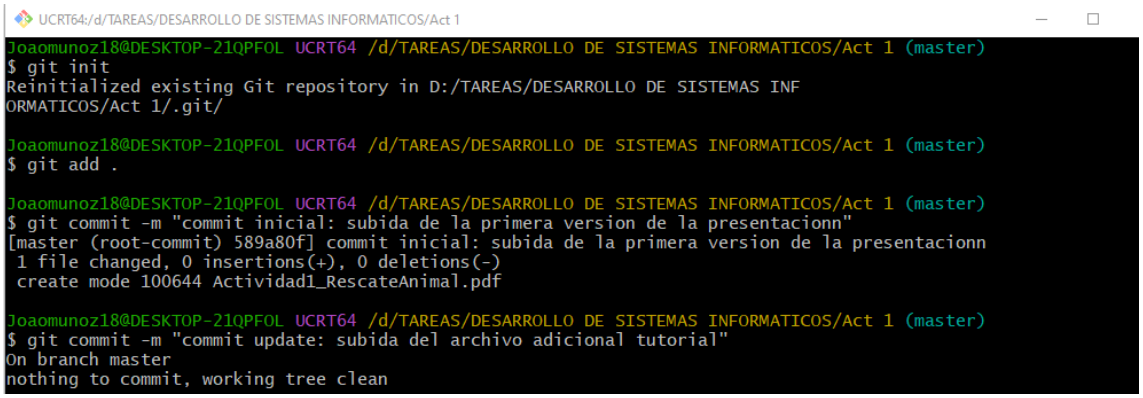
https://github.com/Joaomunoz6552/Actividad1-U1

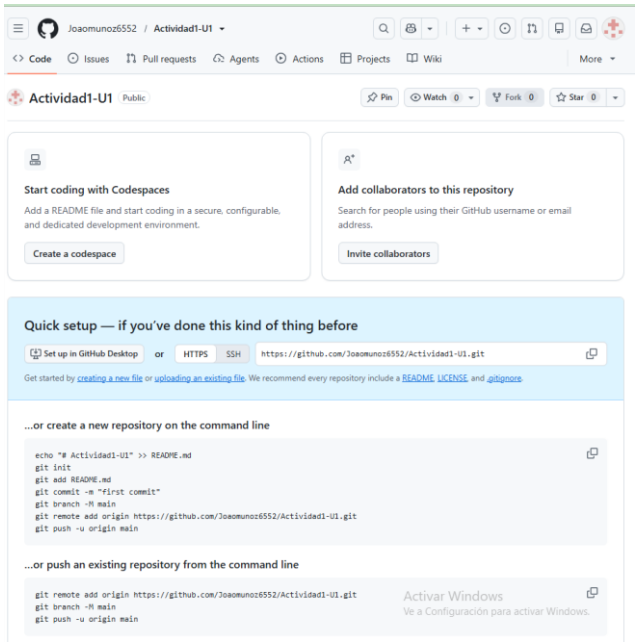
Capturas del proceso realizado

Fase 1: Preparación

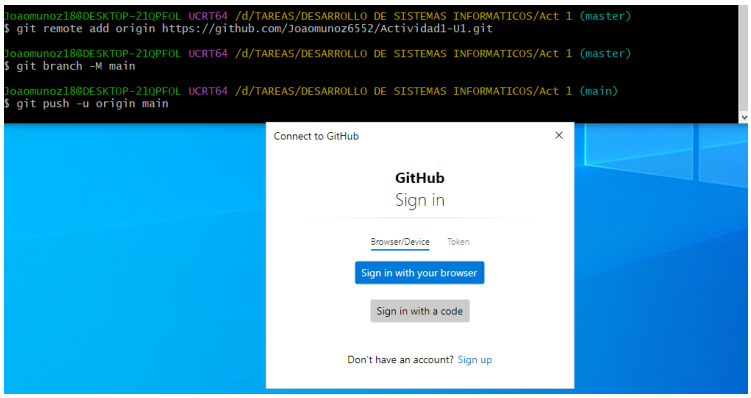


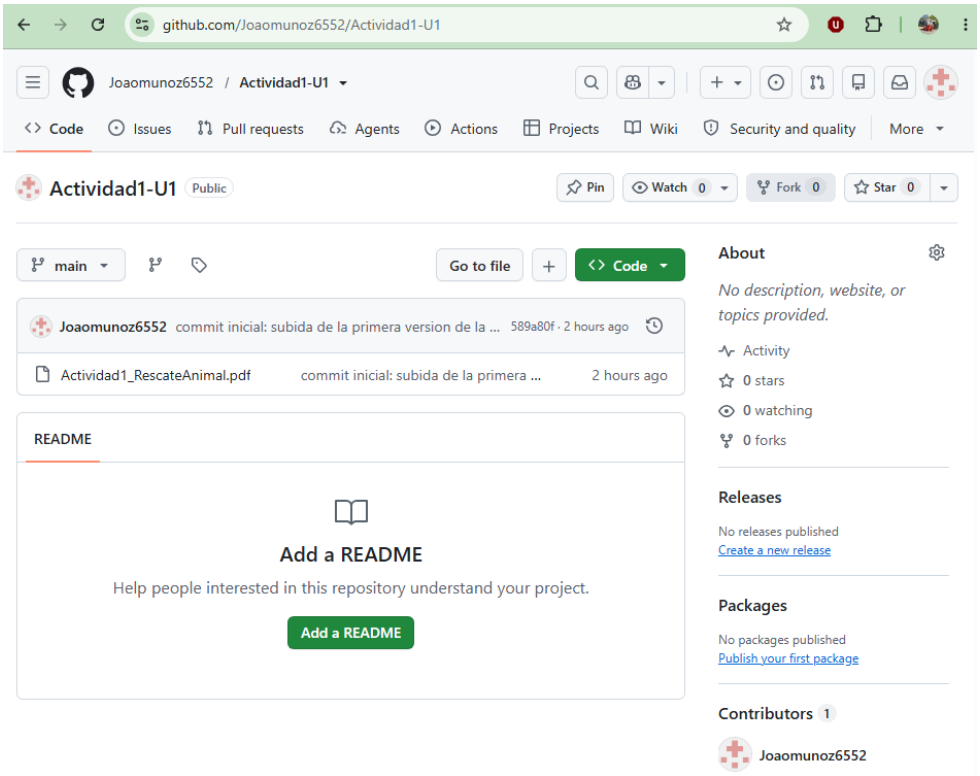
Fase 2: Creación del Proyecto (El “Aplicativo”)

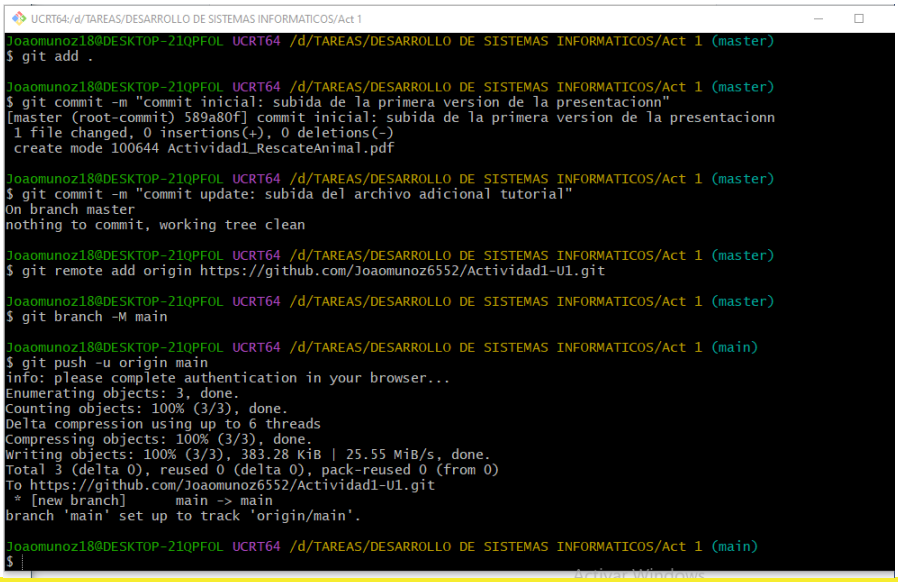




Fase 3: Subir a GitHub (La Nube)







Fase 4: El Ciclo de Actualización

Bibliografía

Atlassian. (s. f.). *Gitflow workflow*. https://www.atlassian.com/en/git/tutorials/comparing-workflows/gitflow-workflow

Refactoring.Guru. (s. f.-a). *Factory Method*. https://refactoring.guru/es/design-patterns/factory-method

Refactoring.Guru. (s. f.-b). *Observer*. https://refactoring.guru/es/design-patterns/observer

Refactoring.Guru. (s. f.-c). *Singleton*. https://refactoring.guru/es/design-patterns/singleton